מעבדה 1: ארגז הכלים של ה – Linux Shell

הקדמה ומטרות

במעבדה זו נרענן את הידע בעבודה בסביבת Linux ובכתיבת סקריפטים ב – Bash. ה – Shell הוא הממשק הישיר שלנו עם מערכת ההפעלה, והבנתו הכרחית לניהול תהליכים, קבצים ואוטומציה של משימות בקורס.

מטרות המעבדה

- עבודה עם פקודות ניווט, ניהול קבצים והרשאות.
- הבנת מבנה סקריפט :Bash שימוש במשתנים, ארגומנטים ומבני בקרה.
- שליטה ב Pipelines-ועיבוד טקסט מתקדם.(grep, cut, sort, uniq)

התקנת סביבת עבודה

השלימו את התקנת סביבת העבודה בה תעבדו (WSL, שרת המכללה או מכונה וירטואלית) לפי מסמך ההנחיות הנמצא במודל "התקנת סביבת עבודה".

וודאו כי סביבת העבודה עובדת ע"י הרצת שורת הפקודה הבאה: "echo "OS is challenging course

מבנה הנתונים והקבצים

למעבדה זו מצורפים שני קבצים המדמים נתונים ממערכת אמיתית:

1. $\boldsymbol{raw_data.txt}$: קובץ לוג (Log) גולמי. כל שורה מייצגת אירוע במערכת במבנה הבא

   $[User_Name]\ [Action_Type]\ [Module]\ ID: [Value]$

   - דוגמה: $Alice\ ERROR\ DB\ ID: 106$

2. $\boldsymbol{users.txt}$: קובץ טקסט המכיל רשימת משתמשים. העמודות מופרדות ברווחים:

   $[Name]\ [Role]\ [Department]\ [ID]\ [LastLogin]$

   - דוגמה: $Sybil\ Manager\ Marketing\ 100001\ 2024-02-01$

דגשים והסברים (למידה עצמית ודוגמאות)

חלק זה נועד לרענון. עברו על הדוגמאות והריצו אותן בסביבת העבודה שלכם לפני שתתחילו במשימות.

משתנים וארגומנטים בסקריפט

- סקריפט Bash מתחיל ב - $\#!/bin/bash$
- $\$1, \$2$ הם הארגומנטים מהמשתמש. $\#$ הוא מספר הארגומנטים.
- ניתן לשמור פלט של פקודה בתוך משתנה כך $MY_VAR = \$(command)$
- <u>פקודת התנאי []</u>: הסוגריים המרובעים הם למעשה קיצור לפקודה שנקראת $test$. לכן, **חובה** להשאיר רווח אחרי הסוגרי הפותח ולפני הסוגרי הסוגר.
- <u>השמה למשתנים</u>: כשמציבים ערך למשתנה, **אסור** שיהיה רווח מסביב לסימן השווה (=)
- השוואת מספרים לעומת מחרוזות:
  - למספרים נשתמש בקיצורים $-eq$ (שווה), $-gt$ (גדול מ) $-lt$(קטן מ)...

- למחרוזות נשתמש בסימנים == ו - = !

דוגמה פתורה 1: יצירת סביבת עבודה (set_env.sh)

הסקריפט מקבל שם תיקייה, יוצר אותה (אם אינה קיימת) ומייצר בתוכה 3 קבצי לוג.

```
#!/bin/bash
if [ $# -lt 1 ]; then
    echo "Usage: $0 <directory_name>"
    exit 1
fi
DIR=$1
if [ ! -d "$DIR" ]; then
    mkdir "$DIR"
    echo "Created directory: $DIR"
fi
for i in {1..3}; do
    touch "$DIR/log_$i.txt"
done
```

עיבוד נתונים ב PIPELINE

- $grep$ – מסנן שורות המכילות תבנית
- cut – חותך עמודות (למשל $cut - d' ' - f1$ יחתוך מילה ראשונה בשורה המופרדת ע"י רווח).
- $sort - n$ – מיון מספרי.
- $sort - r$ – מיון בסדר יורד.

דוגמה פתורה 2: ספירת אירועים ייחודיים

כדי לדעת כמה IDs ייחודיים קיימים בקובץ הלוג.

```
cut -d' ' -f4 raw_data.txt | cut -d':' -f2 | sort | uniq | wc -l
```

חשוב להבין שפקודת $uniq$ בודקת רק אם שורה **זהה לשורה שבאה מיד אחריה**. אם הערך "101" מופיע בשורה הראשונה ושוב בשורה החמישית, וביניהם מופיע ערך אחר, פקודת $uniq$ לא תזהה שהם כפולים ותשאיר את שניהם.

הפתרון: אנחנו מריצים $sort$ קודם. המיון מרכז את כל הערכים הזהים זה אחר זה בבלוקים רציפים, ואז $uniq$ יכולה לזהות אותם בקלות ולמחוק את הכפילויות.

משימות לביצוע

יש לכתוב שלושה סקריפטים. הדיוק בפורמט הפלט קריטי עבור הבודק האוטומטי.

משימה 1: ניתוח אבטחה (security_analyzer.sh)

כתבו סקריפט שמקבל כארגומנט שם של קובץ לוג. למשל, $raw_data.txt$

1. בדיקה אם הקובץ קיים. אם לא, הדפיסו $File\ not\ found!$ וצאו.
2. מצאו את כל השורות המכילות את המילה $ERROR$ או $ATTACK$
3. מתוך שורות אלו, חלצו רק את ה – ID (הערך המספרי אחרי ID:)
4. הדפיסו את ה IDs הייחודיים שנמצאו, **ממוינים מהגדול לקטן**.
5. בשורה האחרונה של הפלט, הדפיסו $Summary: X\ unique\ incidents\ found.$ (X הוא מספר השורות הייחודיות שנמצאו).

משימה 2: ניהול הרשאות משתמשים (**user_filter.sh**)

כתבו סקריפט המעבד את הקובץ $users.txt$

1. סננו רק את המשתמשים ששייכים למחלקת $Engineering$ **וגם** התפקיד שלהם הוא $Admin$
2. שמרו את התוצאה לקובץ חדש בשם $admins_list.csv$ (בפורמט $Name, ID, LastLogin$ – מופרד ע"י פסיקים).
3. שנו את ההרשאות של הקובץ שנוצר כך שרק לבעלים (Owner) תהיה הרשאת קריאה וכתיבה (ללא שום הרשאות לקבוצה ולאחרים – 600).
4. הדפיסו למסך $Filter\ complete.\ File\ permissions\ updated.$

משימה 3: דוח סטטיסטי אישי(**user_stat.sh**)

כתבו סקריפט המשלב מידע משני הקבצים:

1. הסקריפט יקבל שם של משתמש כארגומנט (למשל, $Alice$).
2. הסקריפט יחפש בקובץ $users.txt$ את ה ID של המשתמש. אם המשתמש לא נמצא, הדפיסו $User\ not\ found$ וצאו.
   - **טיפול בכפילויות**: במידה והמשתמש מופיע יותר מפעם אחת עם מספרי ID שונים, יש לקחת את ה-ID שמופיע **ראשון** בקובץ.
3. הסקריפט יספור כמה פעמים מופיע ה ID הזה בקובץ $raw_data.txt$ תחת אירועי $ERROR$
4. **הפלט חייב להיות בשורה אחת בדיוק כך**

$$User: [Name] \mid ID: [Number] \mid Total\ Errors: [Count]$$

מה להגיש ובאיזה פורמט

יש להגיש קובץ ZIP בשם $Lab1_ID.zip$ (החליפו את ID בתעודת הזהות שלכם) על הקובץ להכיל את הקבצים:

1. security_analyzer.sh
2. user_filter.sh
3. user_stat.sh

<u>דגשים להגשה:</u>

- יש לוודא שלכל הקבצים יש הרשאות ריצה.(chmod +x)
- הבודק האוטומטי משווה פלטים בצורה מדויקת .(Case Sensitive) אין להדפיס הודעות נוספות מעבר לנדרש.
- וודאו שכל סקריפט מתמודד נכון עם ארגומנטים חסרים.